

# **TRƯỜNG ĐẠI HỌC MỎ - ĐỊA CHẤT**



**THỊ GIÁC MÁY**

**BÀI TẬP THỊ GIÁC MÁY**

**Sinh viên thực hiện: Nguyễn Hoàng Long**

**MSSV: 2321060041**

**Lớp: DCCDTD68B**

**Giảng viên hướng dẫn: GVC.TS Đoàn Công Luận**

**Hà Nội, 03/2026**

## MỤC LỤC

|                                  |    |
|----------------------------------|----|
| DANH MỤC HÌNH ẢNH.....           | 2  |
| CÂU 1.....                       | 3  |
| 1. Mục đích.....                 | 3  |
| 2. Các cơ sở lý thuyết .....     | 3  |
| 3. Kết quả chạy .....            | 3  |
| 4. Đoạn code thực thi chính..... | 7  |
| CÂU 2.....                       | 10 |
| 1. Phương pháp.....              | 10 |
| 2. Đoạn code thực thi chính..... | 10 |
| 3. Kết quả chạy .....            | 11 |

## DANH MỤC HÌNH ẢNH

|                                                          |    |
|----------------------------------------------------------|----|
| Hình 1. Ảnh gốc .....                                    | 3  |
| Hình 2. Ảnh Filter 1D trung bình theo chiều ngang.....   | 4  |
| Hình 3. Ảnh Filter 1D có trọng số theo chiều ngang ..... | 4  |
| Hình 4. Ảnh Filter 1D trung bình theo chiều dọc.....     | 5  |
| Hình 5. Ảnh Filter 1D có trọng số theo chiều dọc .....   | 5  |
| Hình 6. Ảnh Filter 2D trung bình.....                    | 6  |
| Hình 7. Ảnh Filter 2D có trọng số.....                   | 6  |
| Hình 8. Ảnh gốc .....                                    | 11 |
| Hình 9. Ảnh làm mờ.....                                  | 11 |
| Hình 10. Ảnh dùng Canny Edge lấy biên.....               | 12 |
| Hình 11. Ảnh sau khi ghép lại .....                      | 12 |
| Hình 12. Ảnh so sánh .....                               | 13 |

# CÂU 1

## 1. Mục đích

Khảo sát tác động của các filter 1D và 2D trong việc làm mịn và nhiễu ảnh

## 2. Các cơ sở lý thuyết

Các tích chập được sử dụng trong bài:

$$\text{Filter 1D trung bình (ngang): } H_{avg\_row} = \frac{1}{5} [1 \ 1 \ 1 \ 1 \ 1]$$

$$\text{Filter 1D có trọng số (ngang): } H_{widht\_row} = \frac{1}{10} [1 \ 2 \ 4 \ 2 \ 1]$$

$$\text{Filter 1D (dọc): } H_{avg\_column} = H_{avg\_row}^T \text{ và } H_{width\_column} = H_{width\_row}^T$$

$$\text{Filter 2D trung bình } 3 \times 3 : H_{2d\_avg} = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$\text{Filter 2D trung bình } 3 \times 3 : H_{2d\_avg} = \frac{1}{20} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 8 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

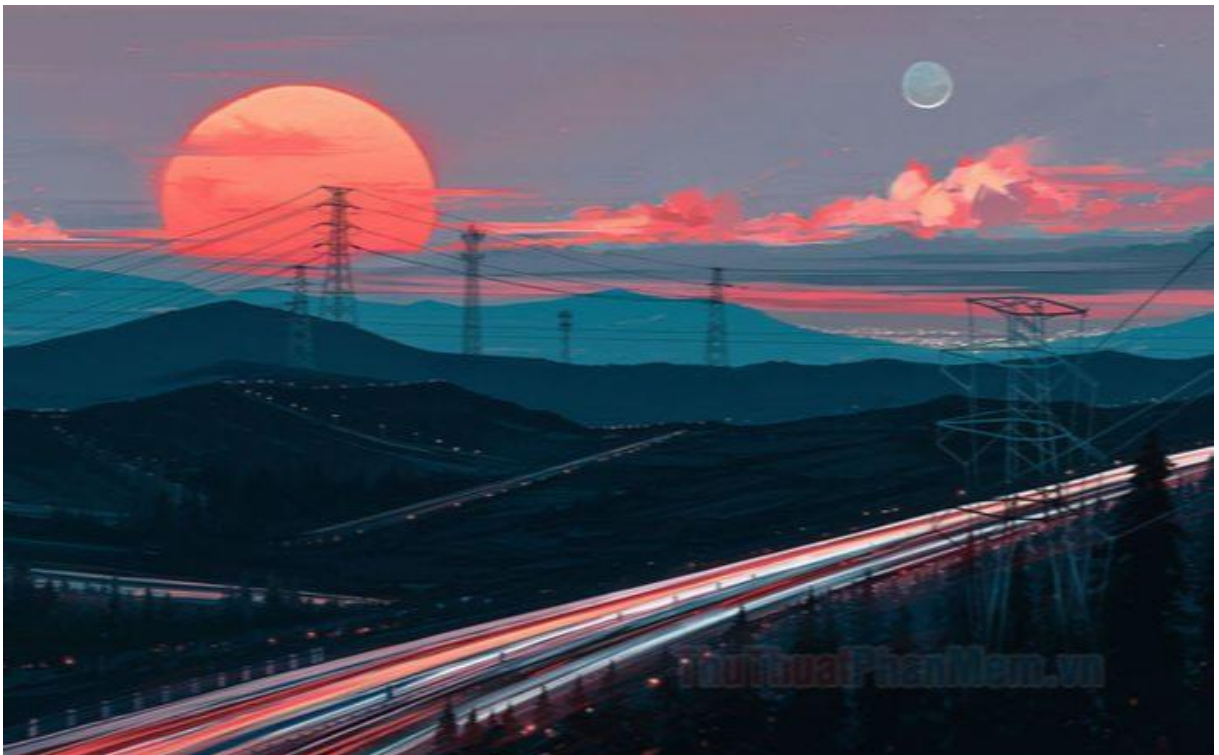
## 3. Kết quả chạy



Hình 1. Ảnh gốc



Hình 2. Ảnh Filter 1D trung bình theo chiều ngang



Hình 3. Ảnh Filter 1D có trọng số theo chiều ngang



Hình 4. Ảnh Filter 1D trung bình theo chiều dọc



Hình 5. Ảnh Filter 1D có trọng số theo chiều dọc



Hình 6. Ảnh Filter 2D trung bình



Hình 7. Ảnh Filter 2D có trọng số

## 4. Đoạn code thực thi chính

```
from __future__ import annotations

import argparse
from pathlib import Path

from src.application.spatial_smoothing_use_case import ExecutionConfig, SpatialSmoothingUseCase
from src.infrastructure.image_io import ImageIO
from src.infrastructure.opencv_filter_engine import OpenCVFilterEngine
from src.infrastructure.workspace_image_provider import WorkspaceImageProvider
```

```
def main() -> None:
    args = parse_args()

    use_case = SpatialSmoothingUseCase(
        filter_engine=OpenCVFilterEngine(),
        image_io=ImageIO(),
        image_provider=WorkspaceImageProvider(seed=args.seed),
    )

    outputs = use_case.execute(
        ExecutionConfig(
            input_path=args.input,
            search_root=args.search_root,
            output_dir=args.output_dir,
            show_windows=args.show,
        )
    )

    print("Spatial smoothing pipeline completed.")
    for name, path in outputs.items():
        print(f"- {name}: {path}")

if __name__ == "__main__":
    main()
```

```
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

import cv2
import numpy as np

from src.domain.contracts import FilterEngine
from src.domain.kernels import KernelFactory
from src.infrastructure.image_io import ImageIO
from src.infrastructure.workspace_image_provider import WorkspaceImageProvider

@dataclass(frozen=True)
class ExecutionConfig:
    input_path: Path | None
    search_root: Path
    output_dir: Path
    show_windows: bool = False
```

```

class SpatialSmoothingUseCase:
    """Execute all assignment-required smoothing operations."""

    def __init__(
        self,
        filter_engine: FilterEngine,
        image_io: ImageIO,
        image_provider: WorkspaceImageProvider,
    ) -> None:
        self._filter_engine = filter_engine
        self._image_io = image_io
        self._image_provider = image_provider

    def execute(self, config: ExecutionConfig) -> dict[str, Path]:
        selected_input = self._resolve_input_path(config)
        source_image = self._image_io.load(selected_input)

        mean_1d_horizontal = KernelFactory.mean_1d_horizontal()
        weighted_1d_horizontal = KernelFactory.weighted_1d_horizontal()

        mean_1d_vertical = mean_1d_horizontal.vertical(name="1d_mean_5x1_vertical")
        weighted_1d_vertical = weighted_1d_horizontal.vertical(name="1d_weighted_5x1_vertical")

        mean_2d = KernelFactory.mean_2d()
        weighted_2d = KernelFactory.weighted_2d()

        results: dict[str, np.ndarray] = {
            "01_original_color": source_image,
            "02_1d_mean_1x5_horizontal": self._filter_engine.apply(
                source_image, mean_1d_horizontal.matrix
            ),
            "03_1d_weighted_1x5_horizontal": self._filter_engine.apply(
                source_image, weighted_1d_horizontal.matrix
            ),
            "04_1d_mean_5x1_vertical": self._filter_engine.apply(
                source_image, mean_1d_vertical.matrix
            ),
            "05_1d_weighted_5x1_vertical": self._filter_engine.apply(
                source_image, weighted_1d_vertical.matrix
            ),
            "06_2d_mean_3x3": self._filter_engine.apply(source_image, mean_2d.matrix),
            "07_2d_weighted_3x3": self._filter_engine.apply(source_image, weighted_2d.matrix),
        }

        output_paths: dict[str, Path] = {}
        config.output_dir.mkdir(parents=True, exist_ok=True)

        for name, image in results.items():
            output_path = config.output_dir / f"{name}.png"
            self._image_io.save(output_path, image)
            output_paths[name] = output_path

            if config.show_windows:
                self._image_io.show(name, image)

        summary_path = config.output_dir / "run_summary.txt"
        summary_path.write_text(
            self._build_summary_text(selected_input=selected_input, outputs=output_paths),
            encoding="utf-8",
        )
        output_paths["run_summary"] = summary_path

        if config.show_windows:
            cv2.waitKey(0)
            cv2.destroyAllWindows()

```

```

def _resolve_input_path(self, config: ExecutionConfig) -> Path:
    if config.input_path is not None:
        return config.input_path

    images = self._image_provider.list_images(config.search_root)
    return self._image_provider.pick_random(images)

@staticmethod
def _build_summary_text(selected_input: Path, outputs: dict[str, Path]) -> str:
    lines = [
        f"Ảnh đầu vào được chọn: {selected_input}",
        "",
        "Các file kết quả:",
    ]
    for key, path in outputs.items():
        lines.append(f"- {key}: {path}")

    lines.extend(
        [
            "",
            "Ma trận lọc đã dùng:",
            "- 1D trung bình (ngang): [1/5, 1/5, 1/5, 1/5, 1/5]",
            "- 1D có trọng số (ngang): [1, 2, 4, 2, 1] / 10",
            "- 1D trung bình (dọc): chuyển vị từ ma trận lọc ngang",
            "- 1D có trọng số (dọc): chuyển vị từ ma trận lọc ngang",
            "- 2D trung bình 3x3: ones(3x3) / 9",
            "- 2D có trọng số 3x3: [[1,2,1],[2,8,2],[1,2,1]] / 20",
        ]
    )

    return "\n".join(lines)

```

```

class KernelFactory:
    """Factory for predefined kernels required by the assignment."""

    @staticmethod
    def mean_1d_horizontal() -> Kernel:
        matrix = np.ones((1, 5), dtype=np.float32) / 5.0
        return Kernel(name="1d_mean_1x5_horizontal", matrix=matrix)

    @staticmethod
    def weighted_1d_horizontal() -> Kernel:
        matrix = np.array([[1, 2, 4, 2, 1]], dtype=np.float32)
        matrix = matrix / matrix.sum()
        return Kernel(name="1d_weighted_1x5_horizontal", matrix=matrix)

    @staticmethod
    def mean_2d() -> Kernel:
        matrix = np.ones((3, 3), dtype=np.float32) / 9.0
        return Kernel(name="2d_mean_3x3", matrix=matrix)

    @staticmethod
    def weighted_2d() -> Kernel:
        matrix = np.array(
            [
                [1, 2, 1],
                [2, 8, 2],
                [1, 2, 1],
            ],
            dtype=np.float32,
        )
        matrix = matrix / matrix.sum()
        return Kernel(name="2d_weighted_3x3", matrix=matrix)

```

## CÂU 2

### 1. Phương pháp

Dùng Canny Edge detection phát hiện biên trong ảnh gốc, kết quả thu được sẽ là một ma trận nhị phân là 1 tương ứng với pixel 255 là đường biên, 0 là pixel 0 là vùng nền phẳng

Nguyên lý hoạt động:

Bước 1: Sử dụng Canny Edge Detection để tìm ma trận

Bước 2: Áp dụng Filter để làm mịn lên toàn bộ ảnh gốc

Bước 3: Sử dụng ma trận ở bước 1 để tổng hợp ảnh, tại các đường biên thì giữ nguyên, tại các giá trị 0 thì lấy pixel từ ảnh đã làm mờ để khử nhiễu

### 2. Đoạn code thực thi chính

```
def main() -> None:
    args = parse_args()

    use_case = SharpeningUseCase(
        image_io=ImageIO(),
        image_provider=WorkspaceImageProvider(seed=args.seed),
    )

    outputs = use_case.execute(
        SharpeningConfig(
            input_path=args.input,
            search_root=args.search_root,
            output_dir=args.output_dir,
            show_windows=args.show,
            blur_kernel_size=args.blur_kernel,
            blur_sigma=args.blur_sigma,
            canny_low_threshold=args.canny_low,
            canny_high_threshold=args.canny_high,
        )
    )

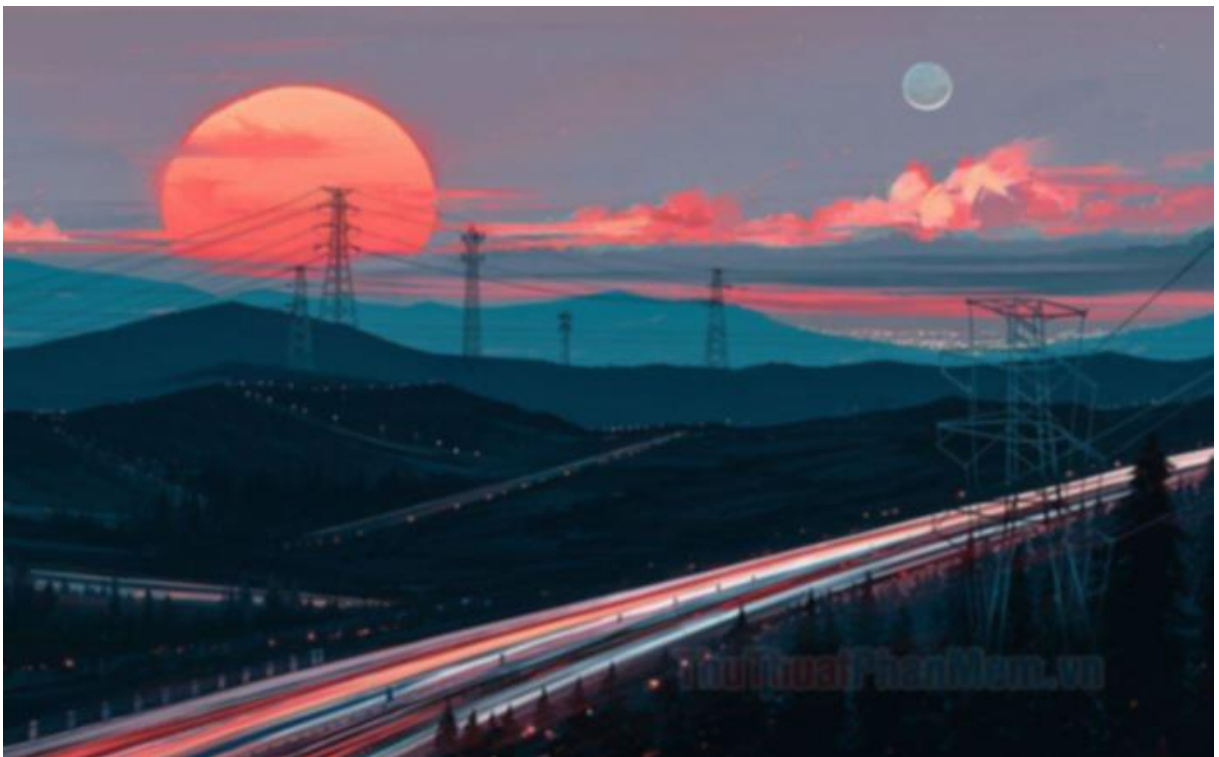
    print("Câu 2 hoàn thành.")
    for name, path in outputs.items():
        print(f"- {name}: {path}")

if __name__ == "__main__":
    main()
```

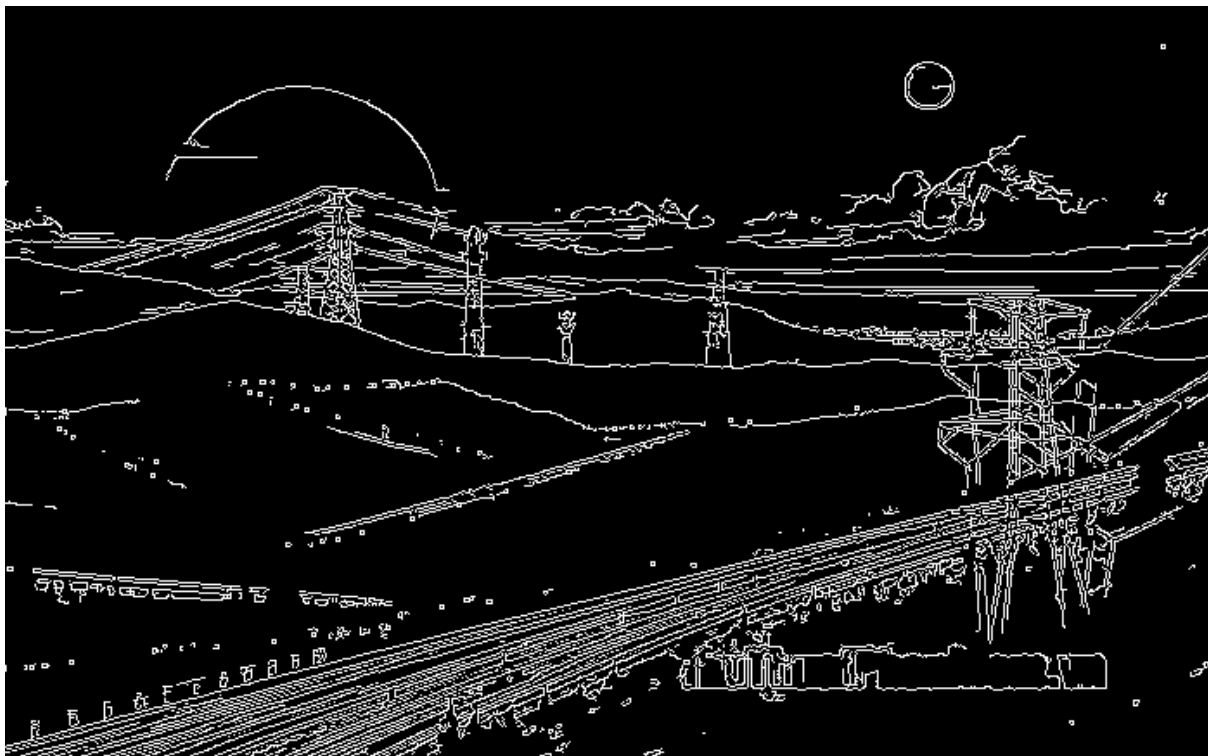
### 3. Kết quả chạy



Hình 8. Ảnh gốc



Hình 9. Ảnh làm mờ



Hình 10. Ảnh dùng Canny Edge lấy biên



Hình 11. Ảnh sau khi ghép lại



*Hình 12. Ảnh so sánh*

**Nhận xét:** So với các Filter ở Câu 1, phương pháp kết hợp này cho ra kết quả ổn hơn hẳn. Các vùng nền nhiễu đã được làm mịn trơn tru, trong khi các cấu trúc hình học, góc cạnh hay đường viền của vật thể không hề bị suy giảm chất lượng.